Runbook — Confluent Platform + Flink on Minikube

End-to-end operational guide for running `confluent-kafka-isotope` locally on **Minikube**: cluster → Kafka/Confluent Platform → Flink → SQL reports → traffic → observe → teardown. Every step maps to a target in the Makefile, which is the source of truth.

> This is the **Confluent Platform + Flink (self-managed) on Minikube** path. The Confluent Cloud for Apache Flink (CCAF) path is entirely different — Terraform-driven via `make cc-flink-reports-up`, no Minikube — see root README §3.4.

Table of Contents

1.0 Prerequisites (once per machine)

```
make install-prereqs     # docker, kubectl, minikube, helm, gettext,
gradle, openjdk17
make check-prereqs       # verify they're on PATH
```

Default Minikube sizing (override via env): `MINIKUBE_CPUS=6`, `MINIKUBE_MEM=20480`, `MINIKUBE_DISK=50g`. The node architecture is auto-detected so the right `cp-flink` image (amd64/arm64) is selected.

2.0 Cluster + Confluent Platform

```
make cp-up               # = check-prereqs → minikube-start → operator-
install → cp-deploy
make cp-watch            # watch pods come up (Ctrl+C to exit); or: make
cp-status
```

`cp-up` boots Minikube, installs the CFK (Confluent for Kubernetes) operator, and deploys Kafka (KRaft) + Schema Registry + Connect + ksqlDB + REST Proxy + Control Center into the `confluent` namespace.

> `cp-up` deliberately does **not** bring up Flink — run §3.0 Flink separately.

3.0 Flink

```
make flink-up             # cert-manager → operator → MinIO → CMF 2.4 → env
→ RBAC → app image
make cmf-status           # (~5 min the first time)
```

This installs cert-manager, the Confluent Flink Kubernetes Operator, **MinIO** (S3-compatible store for CMF artifacts), **CMF 2.4** (with artifact management + the writable environment catalog enabled), creates the `dev-local` Flink environment, applies the Flink RBAC, and builds the custom `cp-flink` 2.1 image (`isotope-cp-flink-sql:local`) that bakes in the Kafka + Avro SQL connectors and the S3 filesystem plugin.

> **The reports run as a CMF *Application*, not SQL statements.** CMF's SQL-statement runtime (`io.confluent.flink.FlinkCompiledPlanExecutor`) ships only in the `cp-flink-sql` image, which exists only at **Flink 1.19** — and two reports are `ProcessTableFunction`s, a **Flink 2.x** feature. So all seven reports deploy as a single Flink 2.1 CMF **Application** (entry point `IsotopeReportsJob`), which runs the same `.fql` and surfaces in CMF / Control Center as a managed application. No raw session cluster is created — if you want one for ad-hoc SQL, deploy it separately with `make flink-deploy` (see §7.0 Observe).

> **CMF license.** The `cp-cmf` image ships with a date-locked trial license. Once it expires, the CMF pod `CrashLoopBackOff`s on startup (`LicenseInitiator` throws) and `make flink-up` fails at the CMF readiness wait. To run past expiry, supply a Confluent license via a secret and point `CMF_LICENSE_SECRET` at it:
>
> ```
> kubectl create secret generic confluent-license-for-cmf -n confluent \
>   --from-file=license.txt=/path/to/license.txt   # key MUST be
> license.txt
> make flink-up CMF_LICENSE_SECRET=confluent-license-for-cmf
> ```
>
> Export `CMF_LICENSE_SECRET` in your shell to make plain `make flink-up` pick it up.

4.0 Port-forward Kafka

```
make kafka-pf-up          # localhost:30092 → Kafka, localhost:8081 → SR
```

Prereq for everything the host-run gradle app does — `App.java`'s defaults already point at `localhost:30092` / `localhost:8081`. Stop with `make kafka-pf-down`.

5.0 Deploy the 7 Flink reports (as a CMF Application)

```
make flink-reports-up    # builds the app shadow JAR (:ptf:shadowJar) → pre-creates
                         # 4 source + 7 sink topics → uploads the JAR as a cmf://
                         # artifact (MinIO) → deploys the FlinkApplication → waits RUNNING
```

All seven reports run in **one** Flink 2.1 CMF Application (`isotope-reports`): five pure Flink SQL plus two JAR-backed `ProcessTableFunction`s (`LatencyPercentilesPTF`, `StuckTracePTF`), executed together as a single `StatementSet` by `IsotopeReportsJob`. Sink topics use Apache Flink's `avro-confluent` format — SR-framed Avro, auto-registered on first write — so Control Center renders the rows natively. The application appears in CMF's applications API and Control Center's Flink tab. Drop everything (application + artifact + sink topics) with `make flink-reports-down`.

6.0 Drive traffic (required to see report rows)

All report jobs aggregate over `TUMBLE(event_time, INTERVAL '1' MINUTE)` windows, which only emit when the watermark advances past `window_end`. Spread records across **multiple** windows:

```
# 30 records, 5s apart ≈ 2.5 min of event-time → spans 3+ windows
for i in {1..30}; do ./gradlew :app:run --args="place burst-$i" -q; sleep 5; done
```

Or run the pipeline-position stages, each in its own terminal:

```
./gradlew :app:run --args="place hello"   # origin → orders.placed
./gradlew :app:run --args="enrich"        # orders.placed   → orders.enriched
./gradlew :app:run --args="fulfill"       # orders.enriched → orders.fulfilled
./gradlew :app:run --args="ship"          # terminal consume orders.fulfilled
```

Wait ~90s after the **last** record before checking results — that's the watermark catching up. `stuck_trace_alerts_1m` only fires for a trace that goes ≥60s of event time without a fresh hop; to exercise it, send one record to `orders.placed`, skip the `enrich`/`fulfill` hops, and keep sending unrelated records so the watermark crosses `event_time + 60s`.

7.0 Observe

```
make c3-open             # Control Center — report topics render natively (Avro+SR)
make flink-ui            # Flink job graph / watermarks
```

```
make metrics-up          # optional: Prometheus + Grafana for the stateless
meters
```

The metrics showcase has its own runbook — k8s/monitoring/README.md (and the meter/PromQL reference in docs/metrics.md). Stop the background port-forwards with `make c3-stop` / `make flink-ui-stop` / `make metrics-down`.

Ad-hoc Flink SQL — needs a session cluster (optional). The reports run as a CMF *Application* (§5.0): one compiled job under `execution.target=kubernetes-application`, which **cannot accept ad-hoc SQL**. Interactive querying needs a separate **session cluster**, and neither `flink-up` nor `flink-reports-up` creates one. Deploy it first:

```
make flink-deploy        # one-time: deploy the 'flink-basic' session
cluster (~40s)
make flink-sql           # interactive SQL Client on it, report DDL
preloaded
```

`flink-deploy` applies k8s/base/flink-cluster-deployment.yaml — a `cp-flink` 2.1 session cluster with 8 task slots, whose init containers fetch the Kafka and Avro-SR connector JARs. `flink-sql` then installs the PTF JAR into the JobManager's `/opt/flink/lib` and preloads the DDL from scripts/flink/sql/cp/, so the session opens ready to query:

```
Flink SQL> SHOW TABLES;          -- 4 source tables + 7 report sinks +
views
Flink SQL> SHOW USER FUNCTIONS;  -- latency_percentiles, stuck_trace_ptf
```

Skip `flink-deploy` and `make flink-sql` stops with a message telling you to run it — it will not fall back to the reports Application pod.

> **The SQL Client catalog is its own.** The preloaded DDL lives in that session only, and the running reports Application registers its tables inside the job's own `TableEnvironment` — no shared catalog exists, so `SHOW TABLES` never reflects what the Application is running. Only DDL is preloaded; the report `INSERT INTO` files (`10_`…`70_`) are deliberately excluded, because the SQL Client rejects DML in an init file. Run them by hand only if you mean to — the Application is already writing to those same sink topics, so you would get duplicate rows, not an error.

8.0 Teardown

Pick the depth:

```
make flink-reports-down  # drop reports / views / functions only
make flink-delete        # drop just the 'flink-basic' ad-hoc SQL session
cluster
make metrics-delete      # remove the Prometheus/Grafana showcase (pods +
namespace)
```

```
make flink-down          # Flink cluster + CMF + operator + cert-manager
(includes flink-delete)
make cp-down             # CP + operator (keeps Minikube running)
make confluent-teardown  # everything + stop Minikube
make nuke                # confluent-teardown + minikube-delete +
uninstall-prereqs (factory reset)
```

9.0 Minimal path (no Flink)

To just watch a trace propagate without the Flink SQL reports:

```
make cp-up
make kafka-pf-up
./gradlew :app:run --args="place hello"   # then enrich / fulfill / ship
```

Flink (§3.0 Flink) and the reports (§5.0 Deploy the 7 Flink reports (as a CMF Application)) are only needed for the SQL report topics.

10.0 Troubleshooting

- **Pods stuck `Pending`.** Minikube is under-resourced — raise `MINIKUBE_CPUS` / `MINIKUBE_MEM` and `make minikube-delete && make cp-up`.
- **Flink job submission fails (`services` forbidden).** The supplemental RBAC didn't apply — `make flink-rbac`.
- **No report rows.** Almost always the watermark — see §6.0 Drive traffic (required to see report rows); traffic must span multiple 1-minute windows and you must wait ~90s after the last record.
- **App can't reach Kafka.** `make kafka-pf-up` isn't running, or the forward died — re-run it and confirm `localhost:30092` / `localhost:8081` are live.
- **`make flink-sql` says no session cluster / `SHOW TABLES;` returns an empty set.** Ad-hoc SQL needs the `flink-basic` session cluster — run `make flink-deploy` first, see §7.0 Observe. Confirm with `kubectl get flinkdeployment -n confluent`: `isotope-reports` alone means only the reports Application is up. Note that the report tables of the running Application are never visible to the SQL Client regardless — the catalogs are separate.
- **Control Center's Flink tab is blank.** CMF proxy connectivity — `make cmf-proxy-inject` (and `make cmf-proxy-logs` to debug).
- **`make flink-up` times out waiting for the CMF pod / CMF `CrashLoopBackOff`.** Check `kubectl logs -n confluent -l app.kubernetes.io/name=confluent-manager-for-apache-flink`. If you see `Trial license ... expired` / `LicenseInitiator ... Constructor threw exception`, the image's embedded trial license has expired — supply your own via `CMF_LICENSE_SECRET` — see the CMF license note in §3.0 Flink. Wiping the CMF `PersistentVolumeClaim` does **not** reset it; the expiry is baked into the image.